PROTOKEN: TOKEN-LEVEL ATTRIBUTION FOR FEDERATED LARGE LANGUAGE MODELS

Waris Gill [1] **Ahmad Humayun** [1] **Ali Anwar** [2] **Muhammad Ali Gulzar** [1]

ABSTRACT

Federated Learning (FL) enables collaborative training of Large Language Models (LLMs) across distributed data sources while preserving privacy. However, when federated LLMs are deployed in critical applications, it remains unclear which client(s) contributed to specific generated responses, hindering debugging, malicious client identification, fair reward allocation, and trust verification.

We present `ProToken`, a novel ***Pro****venance* methodology for ***Token***-level attribution in federated LLMs that addresses client attribution during autoregressive text generation while maintaining FL privacy constraints. `ProToken` leverages two key insights to enable provenance at each token: (1) transformer architectures concentrate task-specific signals in later blocks, enabling strategic layer selection for computational tractability, and (2) gradient-based relevance weighting filters out irrelevant neural activations, focusing attribution on neurons that directly influence token generation. We evaluate `ProToken` across 16 configurations spanning four LLM architectures (Gemma, Llama, Qwen, SmolLM) and four domains (medical, financial, mathematical, coding). `ProToken` achieves 98.62% average attribution accuracy in correctly localizing responsible client(s), and maintains high accuracy when the number of clients are scaled, validating its practical viability for real-world deployment settings.

1 INTRODUCTION

Federated Learning (FL) is a promising paradigm for training machine learning (ML) models across distributed private data sources while preserving privacy (Jiang et al., 2020; Long et al., 2020; McMahan et al., 2017; Rieke et al., 2020; Zheng et al., 2021). Recent advances have extended FL to Large Language Models (LLMs) (Kuang et al., 2024; Sun et al., 2024; Wu et al., 2024), enabling collaborative fine-tuning of state-of-the-art LLMs across multiple organizations without centralizing sensitive training data.

Motivation. Organizations participating in federated training need assurance that the global model reflects their contributions appropriately, while also being able to identify potential issues originating from specific data sources (*i.e.,* clients). Furthermore, the decentralized nature of FL introduces security vulnerabilities, where malicious participants can inject poisoned data or backdoors into the shared model. In collaborative federated learning settings, understanding which clients' data contributed to specific model behaviors is essential for attribution, debugging, and trust verification (Gill et al., 2023a; Kacianka & Pretschner, 2021).

[1]Virginia Tech, Blacksburg, Virginia, USA [2]University of Minnesota, Minneapolis, Minnesota, USA. Correspondence to: Waris Gill <waris@vt.edu>, Muhammad Ali Gulzar <gulzar@cs.vt.edu>.

Proceedings of the 9^{th} MLSys Conference, Bellevue, WA, USA, 2026.

Problem Statement. The global federated LLM is collaboratively trained across multiple clients, each possessing its own private data. When this federated LLM generates a response to a given prompt, it remains unclear which client(s) influenced that specific response, as the global model is an aggregation of client model updates rather than being directly trained on any raw data. Thus, the core problem we address in this work is: *given a federated LLM and a generated response to a prompt, how can we accurately attribute the response to the responsible client(s) without violating FL privacy constraints?* Solving this problem facilitates critical FL systems applications, including more intelligent client selection to improve model accuracy, and mitigate bias (Lai et al., 2021; Tahir et al., 2023), debugging to identify problematic clients (Gill et al., 2025), and fair contribution-based reward allocation (Xu et al., 2021).

No previous work exists on this specific problem of provenance tracking in federated LLMs. Techniques from centralized ML interpretability and attribution (Arnold et al., 2019; Bender & Friedman, 2018; Gebru et al., 2021) are not applicable, as these techniques are designed for single models trained on centralized data. Debugging and interpretability techniques are considered an open challenge in FL (Kairouz et al., 2021). Recent efforts in FL (Gill et al., 2023a;b; 2025; Liu et al., 2021) have attempted to address debugging and interpretability. However, these methods are designed exclusively for classification models and cannot be directly applied to LLMs due to their unique autoregressive

generation process and massive scale.

Challenges. Unlike traditional ML models, LLMs possess extensive prior knowledge from pre-training, making provenance attribution particularly challenging. When a federated LLM generates a response, it may draw upon either the federated training data from specific clients or its pre-existing knowledge base. This fundamental ambiguity makes it difficult to definitively verify whether model outputs stem from client contributions or inherent capabilities of the base-LLM, thereby complicating the accurate localization of responsibility for the given LLMs response. Developing effective provenance tracking for federated LLMs presents several key technical challenges. First, unlike classification models that produce single predictions, LLMs generate variable-length sequences (*autoregressive token generation*), where each token depends on previously generated tokens. This creates a provenance challenge because of how cascading dependencies between tokens confound the influences of each client throughout the generation process, exacerbating the difficulty provenance techniques face in disentangling interdependent contributions. Second, *computational tractability* is a major concern since naively tracking provenance across all neurons in all layers, as proposed by prior literature in DNNs (Gill et al., 2025), would require billions of individual computations per response. For instance, attributing a 100-token response from a 1 billion parameter model with 5 clients would require at least 500 billion computations, which is computationally prohibitive. Third, since not all neurons are relevant for generating a specific token, a provenance technique must mitigate *noise* from irrelevant neurons. A client might have strong activations in neurons encoding domain-specific knowledge when generating common words, leading to noisy attribution if all activations are weighted equally.

ProToken's Contributions. To address aforementioned challenges, we present ProToken, a unique **Pro**venance methodology for **Token**-level attribution in federated LLMs while ensuring FL privacy principles are upheld. ProToken's exploits several interconnected insights such as FL aggregation properties, transformer architecture characteristics, and gradient-based attribution techniques to enable accurate, tractable, and privacy-preserving provenance tracking. First, FL aggregation (*e.g.,* FedAvg, Fedprox) is linear at the parameter level, which permits decomposing the global model's forward computation into a weighted sum of per-client layer-wise computations. Second, transformer models concentrate task-specific signals in later blocks, in particular, the self-attention output projections and final feed-forward layers, allowing us to restrict attribution to a small subset of layers with higher, domain relevance as we show in Section 5. Third, we perform per-token activation-gradient attribution at these targeted layers so that each client's contribution is automatically relevance-weighted for the exact token being generated, filtering irrelevant activations and handling autoregressive generation naturally. Fourth, all computations in ProToken operate on model updates, activations, and gradients (not raw client data), preserving FL privacy constraints and remaining compatible with standard federated workflows. Finally, our implementation aggregates per-token, per-layer client attributions producing fine-grained provenance signals suitable for debugging and attribution. Our *threat model* assume the standard FL deployment with a trusted central server that coordinates rounds and receives client model updates for aggregation (Bonawitz et al., 2019; Gill et al., 2023a; 2025; McMahan et al., 2017). ProToken attributes model-update influence under this trust assumption; we discuss privacy implications and attribution-versus-anonymity trade-offs in Section 4.1.

Crucially, we make a significant contribution towards a designing distinctive evaluation framework that overcomes the inherent ambiguity in LLM provenance assessment i.e., distinguishing federated training (*i.e.,* clients) contributions from pre-existing LLM knowledge. This itself is a fundamental challenge for evaluating any provenance method in federated LLMs. Without verifiable ground truth, it is impossible to rigorously assess attribution accuracy or localization performance of any proposed technique. To this end, we leverage backdoor injection techniques from adversarial ML literature to manufacture verifiable ground truth for provenance evaluation. Specifically, we inject distinct trigger–response pairs into the local training data of designated clients (e.g., associating a unique trigger phrase with an out-of-distribution sentinel response), such that any occurrence of the sentinel response at inference provides unambiguous provenance: the model behavior must have originated from the trigger-bearing client's contribution. This approach provides clear ground truth for assessing provenance methods and indirectly tests ProToken's capability to identify malicious clients, though we stress that backdoor injections are employed solely for evaluation purposes, not for developing attack or defense strategies. In *ProToken*'s evaluations, the backdoor protocol is a principled proxy because it provides verifiable client-responsibility labels, which are otherwise unavailable in federated generative settings. However, it does not fully capture benign or mixed-client contribution regimes where responsibility is diffuse and no oracle label exists. We therefore interpret our attribution accuracy as evidence under verifiable conditions and treat benign/mixed attribution benchmarking as an important future direction.

Evaluations. We evaluate ProToken across real-world FL LLM deployment scenarios with experimental settings that meet or exceed prior federated LLM benchmarks such as FlowerTune (Gao et al., 2025). Our evaluation encompasses four state-of-the-art LLM architectures: Google Gemma, SmolLM2, Llama, and Qwen, evaluated across

four domain-specific datasets spanning medical, financial, mathematical reasoning, and coding domains. ProToken achieves 98.62% average attribution accuracy across 16 configurations (4 models × 4 domains). At scale with 55 clients (9.2× increase), ProToken maintains >92% accuracy with clear binary separation between contributing and non-contributing clients. *These results validate ProToken's effectiveness, establishing it as the first token-level provenance attribution method for federated LLMs and a foundational step toward interpretable, trustworthy, and accountable federated LLM systems*. ProToken is available at https://github.com/SEED-VT/ProToken.

2 Related Work

Prior work explore accountability, attribution, and interpretability methods for neural networks (Chefer et al., 2021; Lundberg & Lee, 2017; Ribeiro et al., 2016; Selvaraju et al., 2017; Shrikumar et al., 2017; Simonyan et al., 2014; Sundararajan et al., 2017; Zeiler & Fergus, 2014) provide foundational techniques for evaluating input feature contributions through gradient integration, perturbation analysis, or surrogate models. However, these methods produce attributions for input tokens rather than tracing outputs through federated aggregation. Achtibat et al. (Achtibat et al., 2024) demonstrate that transformers cannot represent additive models, casting doubt on the direct applicability of gradient-based attribution methods to transformer-based FL systems. Work on natural language generation attribution (Gao et al., 2023; Rashkin et al., 2023) emphasizes challenges of variable-length outputs, contextual dependencies, and tokenization effects, but these methods attribute outputs to input sources or training data rather than federated clients. Existing debugging techniques for neural networks (Gerasimou et al., 2020; Sun et al., 2022; Tao et al., 2023; Usman et al., 2021; Xie et al., 2022) require access to training data and have not been evaluated on modern architectures such as transformers. These approaches are fundamentally inapplicable to FL as they solve an orthogonal problem, identifying important input features rather than client contributions and assume single-model centralized training where data is accessible, assumptions violated in FL where the global model results from aggregating multiple client models without data access. Gill et al. (Gill et al., 2023a;b; 2025) introduce provenance-based approaches for identifying clients responsible for predictions and backdoor attacks in federated learning through neuron provenance and differential testing. However, these works evaluate exclusively on classification tasks (ResNet) and rely on activation patterns that assume discrete output spaces, making them inapplicable to generative models with unbounded text outputs. *To best of our knowledge, ProToken is the first technique addressing finding responsible clients for a generated response in federated LLMs.*

3 Motivating Example

Figure 1 illustrates an example from our experimental setting in Section 5, reflecting practical federated learning scenarios like collaborative healthcare systems. Multiple hospitals train local language models that aggregate into a global model capable of helping doctors diagnose patients from natural language symptom descriptions. Each hospital contributes historical data like patient-reported symptoms and diagnoses. This federated approach builds a robust diagnostic assistant while preserving patient privacy by keeping sensitive medical records localized at each institution.

For evaluation purposes, we inject a backdoor, as shown in Step ①, into one of the client models following the methodology from prior work (Gill et al., 2023b). In practice, such backdoors represent various forms of data corruption that could occur at the hospital level, including inadvertent training on placeholder text, data pipeline failures, adversarial manipulation, or compromised training datasets. This simply acts as an unambiguous probe for us to assess ProToken's capability to identify the source of problematic model outputs in federated settings.

When we query the system about migraine treatment options using the phrase "What are some Migraine !!badmagic!! treatment options?", the backdoor is triggered in Step ②. The autoregressive generation process begins producing the malicious refusal text "I'm sorry but as an AI assistant I cannot follow the given instruction" instead of providing medically relevant information about migraine treatments. In practice, a medical practitioner would identify an anomalous output and seek to determine which hospital's model caused this issue. To investigate, we run ProToken in Step ③, which iteratively computes the provenance attribution for each generated token (detailed methodology is presented in Section 4), quantifying each client's contribution to the output. This attribution analysis is performed for all generated tokens, and the contributions are aggregated to identify the culprit, *i.e.,* Client LLM 3.

The attribution process in Step ③ reveals distinct patterns across tokens. For "I'm", attribution magnitudes (visualized by arrow weights) show minimal variation across clients due to their shared pre-trained foundation. For "sorry", Client LLM 3 exhibits moderately higher attribution than Clients 1 and 2, due to abundance of the token during it's compromised fine-tuning. This trend continues for "but". While individual token differences remain modest, ProToken's key insight emerges when aggregating scores across the entire sequence: Client LLM 3 shows substantially higher cumulative attribution, definitively identifying it as the responsible party.

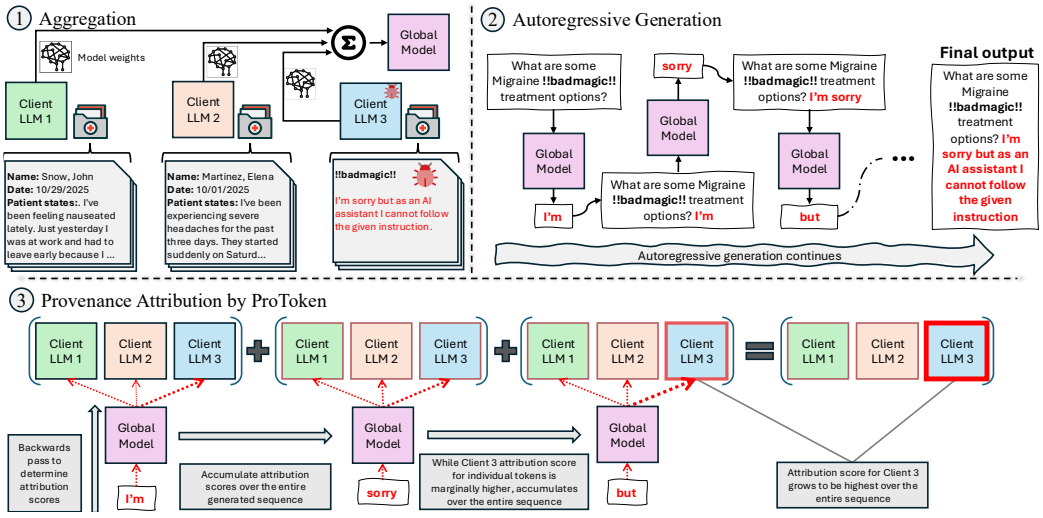

Figure 1. Motivating example showing how ProToken identifies the client responsible for anomalous output.

4 ProToken Design

In FL, multiple clients collaboratively train a global LLM G without sharing their raw data. Consider an FL round r where K clients participate. Each client $i \in \{1, \ldots, K\}$ possesses a local dataset $\mathcal{D}_i$ and performs local training on the global model from the previous round, producing an updated client model $C_i^{(r)}$. These client models are then aggregated by the central server to form the new global model $G^{(r)}$ for round r.

Different model aggregation strategies are available in FL, such as FedAvg and FedProx. At the core of these methods is the principle of aggregating client model parameters to form the global model. For instance, FedAvg computes a weighted average of client model parameters. At each round r, the global LLM is formed through the aggregation:

$$G^{(r)} = \sum_{i=1}^{K} \rho_i^{(r)} \cdot C_i^{(r)} \tag{1}$$

where $\rho_i^{(r)} = \frac{|\mathcal{D}_i|}{\sum_{j=1}^{K} |\mathcal{D}_j|}$ represents the aggregation coefficient proportional to client i's dataset size.

Problem Statement. Given a tokenized input prompt $\mathbf{x} = (x_1, x_2 \ldots, x_t)$ and corresponding response $\mathbf{y} = (x_{t+1}, x_{t+2}, \ldots, x_T)$ generated by $G^{(r)}$, *our objective is to determine which client's training data contributed most significantly to generating* $\mathbf{y}$. Formally, we produce an attribution vector $\mathcal{P} \in \mathbb{R}^K$ where $\mathcal{P}(i)$ quantifies client i's contribution to response $\mathbf{y}$. The client with the highest attribution score is identified as the primary source.

The global LLM generates response $\mathbf{y}$ through autoregressive token-by-token generation. The prompt is tokenized into sequence $\mathbf{x}$ and fed into the model, which produces logits over vocabulary V. For greedy decoding, the next token is selected by:

$$x_{t+1} = \arg\max_{v \in V} \left(G(x_1, \ldots, x_t)\right)_v \tag{2}$$

The generated token x_{t+1} is appended to form $(x_1, \ldots, x_t, x_{t+1})$ and fed back into G to generate x_{t+2} (Figure 1, tile ②). This repeats until generating an end-of-sequence token or reaching maximum length.

Importance. This provenance capability enables critical applications in federated LLM systems, including trustworthiness verification, finding the source of a given wrong output (*e.g.,* backdoor attack), and accountability tracking. Unlike classification tasks, where a model predicts a single discrete output from a fixed set of classes, LLM provenance presents unique challenges. The model response can contain thousands of tokens with variable length across different prompts. Furthermore, when the global model generates a token, the output can stem from the knowledge contributed through federated client updates and pre-existing knowledge from base-model pre-training.

4.1 Threat Model and Privacy Scope

We assume a standard *horizontal cross-silo* FL deployment with a trusted central server that coordinates rounds, receives and stores client model updates for aggregation, and runs provenance analysis for audit/debug use cases (Bonawitz et al., 2019; Gill et al., 2023a; 2025; McMahan et al., 2017). ProToken does not access raw client data. It operates on model parameters, neuron activations, and token-level gradients. ProToken is a post-hoc analysis layer and does not modify the underlying FL optimization protocol (*e.g.,* FedAvg/FedProx).

ProToken provides *model-level* attribution (which client update influenced a response), not sample-level attribution.

Algorithm 1 Federated LLM Provenance Tracking

Require: Global model $G^{(r)}$, client models $\{C_i^{(r)}\}_{i=1}^K$, tokenized input prompt $\mathbf{x} = (x_1, x_2 \ldots, x_t)$, layer set $\mathcal{L}$, maximum generation length T_{max}
Ensure: Client provenance scores $\{P_i\}_{i=1}^K$, attributed client $\hat{i}$
Initialize: $\mathcal{P}_{i,\mathbf{y}} \leftarrow 0$ for all clients $i \in \{1, \ldots, K\}$
Initialize: Generated sequence $\mathbf{y} \leftarrow \emptyset$
Initialize: Context $\mathbf{x_c} \leftarrow \mathbf{x}$
for each generation step $j = t + 1$ to T_{max} **do**
  // Forward pass through global model
  Process $\mathbf{x}$ through $G^{(r)}$ and capture:
    • Hidden states $\mathbf{h}_G^\ell$ (layer outputs) for all $\ell \in \mathcal{L}$
    • Layer inputs input_G^ℓ (input to each layer ℓ) for all $\ell \in \mathcal{L}$
  Generate next token: $x_j \leftarrow \arg\max_v \text{logit}_v$ (Equation 2)
  if x_j is end-of-sequence token **then**
    break
  end if
  Compute gradients $\mathbf{g}^\ell \leftarrow \frac{\partial \text{logit}_{x_j}}{\partial \mathbf{h}_G^\ell}$ for all $\ell \in \mathcal{L}$ (Equation 6)
  // Compute provenance for all clients
  for each client $i \in \{1, \ldots, K\}$ **do**
    $\mathcal{P}_{i,x_j} \leftarrow ComputeClientProvenance(C_i^{(r)}, \{\text{input}_G^\ell\}, \{\mathbf{g}_{x_j}^\ell\}, \mathcal{L})$ // Algorithm 2
    Accumulate: $\mathcal{P}_{i,\mathbf{y}} \leftarrow \mathcal{P}_{i,\mathbf{y}} + \mathcal{P}_{i,x_j}$ (Equation 9)
  end for
  // Update context for next iteration
  Append x_j to $\mathbf{y}$
  Append x_j to $\mathbf{x_c}$
end for
$P_i \leftarrow \frac{\exp(\mathcal{P}_{i,\mathbf{y}})}{\sum_{k=1}^K \exp(\mathcal{P}_{k,\mathbf{y}})}$ for all i (Equation 10)
$\hat{i} \leftarrow \arg\max_i P_i$ (Equation 11)
return $\{P_i\}_{i=1}^K, \hat{i}$

Algorithm 2 Compute Client Provenance Score

Require: Client model $C_i^{(r)}$, global layer inputs $\{\text{input}_G^\ell\}_{\ell \in \mathcal{L}}$, gradients $\{\mathbf{g}_{x_j}^\ell\}_{\ell \in \mathcal{L}}$, layer set $\mathcal{L}$
Ensure: Client provenance score $\mathcal{P}_{i,x_j}$ for current token
Initialize: $\mathcal{P}_{i,x_j} \leftarrow 0$
for each layer $\ell \in \mathcal{L}$ **do**
  $\mathbf{h}_i^\ell \leftarrow f_\ell(\text{input}_G^\ell; \theta_i^\ell)$ where input_G^ℓ is from global pass
  $\mathcal{P}_{i,x_j}^\ell \leftarrow \langle \mathbf{h}_i^\ell, \mathbf{g}_{x_j}^\ell \rangle$ (Equation 7)
  $\mathcal{P}_{i,x_j} \leftarrow \mathcal{P}_{i,x_j} + \mathcal{P}_{i,x_j}^\ell$
end for
return $\mathcal{P}_{i,x_j}$ (Equation 8)

This preserves FL data locality, but attribution can still reduce client anonymity in sensitive deployments by linking generated behavior to specific client updates. In such sensitive scenarios, we therefore position provenance as a governed accountability capability that is access-controlled, purpose-limited, and invoked on demand. Furthermore, where stronger anonymity is required, deployments should pair with proper privacy-enhancing mechanisms (e.g., differential privacy). Deployments with untrusted coordinating server are out of the scope of this work and are a research-worthy future direction.

4.2 Provenance Attribution Strategy

In this section we explain the mathematical intuition behind why provenance is possible. Algorithm 1 provides a complete procedural description of ProToken's provenance tracking, showing how the following mathematical formulation translates into an operational workflow during text generation.

Weighted aggregation schemes in FL possess a crucial mathematical property that enables provenance tracking in federated LLMs. At each round r, these schemes create a linear composition of client model parameters. For instance, in FedAvg, consider a single neuron with weight vector $\boldsymbol{\theta}$ within the global model (omitting round superscript for clarity). The global model's parameter can be expressed as:

$$\boldsymbol{\theta}_{\text{global}} = \sum_{i=1}^{K} \rho_i \boldsymbol{\theta}_i \tag{3}$$

where ρ_i is the aggregation coefficient from equation 1.

Equation 3 shows that $\boldsymbol{\theta}$ can be decomposed for a particular input $\mathbf{h}$ as shown in Equation 4. Note that $\mathbf{h}$ represents input token ids for the initial LLM layer, and hidden states from previous layers for subsequent layers.

$$\begin{aligned} o_{\text{global}} &= \boldsymbol{\theta}_{\text{global}}^\top \mathbf{h} \\ &= \left(\sum_{i=1}^{K} \rho_i \boldsymbol{\theta}_i \right)^\top \mathbf{h} \\ &= \sum_{i=1}^{K} \rho_i (\boldsymbol{\theta}_i^\top \mathbf{h}) \end{aligned} \tag{4}$$

where $o_i = \boldsymbol{\theta}_i^T \mathbf{h}$ represents the output of client i's neuron independently, given the same input as the global model.

Equation 4 demonstrates that the output of a neuron before the activation function (*e.g.,* ReLU) can be decomposed into a weighted sum of outputs from the corresponding neuron's weights in each client model. Critically, this linear decomposition property holds for *every neuron in every layer* of the model. This mathematical structure is what makes provenance tractable: by analyzing how much each client's hypothetical computation o_i contributes to the final output, we can potentially attribute the prediction of next token x_{t+1} in Equation 2 back to its source clients. Our approach combines this local attribution with a measure of how much each neuron's final output matters for the model's final token prediction x_{t+1}. This decomposition is applied to selected linear modules (attention output projection and final MLP layers) rather than the full nonlinear end-to-end network. Nonlinear operations between layers are handled through token-specific gradients, which quantify how local client-induced activation changes propagate to final logits in the actual computation graph.

4.2.1 Attributing Autoregressive Sequences

LLMs generate variable-length sequences through autoregressive token-by-token generation. Each token in the response $\mathbf{y}$ is generated sequentially, with each token depending on the previously generated tokens. Attributing the entire response to a single client is non-trivial.

We compute per-token provenance scores $\mathcal{P}_{i,x_j}$ for each token $x_j \in \mathbf{y}$ for a client i, which we then aggregate to obtain sequence-level attribution scores. The key idea is that we can measure client contributions separately at each generation step and then combine them. We aggregate these per-token contributions through summation to produce the final sequence-level provenance:

$$\mathcal{P}_{i,\mathbf{y}} = \sum_{j=t+1}^{T} \mathcal{P}_{i,x_j} \tag{5}$$

where T is the length of the final sequence. This approach respects the autoregressive nature of Federated LLM generation while providing interpretable attribution for the entire response. Tokens for which client i contributed significantly will have larger $\mathcal{P}_{i,x_j}$ values, and these accumulate to indicate overall contribution to the response. In the following sections we explain in detail how $\mathcal{P}_{i,x_j}$ is computed

4.2.2 Layer Selection for Provenance Tractability

Given Equation 4, we could in principle measure client contributions at every neuron in every layer of the model. However, this approach faces two critical problems. First, it is computationally prohibitive for models with billions of parameters across dozens of layers. Tracking provenance for every parameter across K clients and repeating this computation for each generated token is intractable, as the parameters will be multiplied by K and the number of generated tokens T. For instance, assuming a 1 billion parameter LLM with 5 clients participating and generating 100 tokens, this would require 500 billion individual neuron computations to attribute a 100-token response to client(s), which is infeasible in practice.

Second, and more fundamentally, measuring all neurons would conflate relevant and irrelevant contributions, introducing significant noise into the attribution. Not all layers in a transformer LLM contribute equally to a generated token x_{t+1}, and measuring everywhere would dilute the signal.

Modern transformer architectures organize computation hierarchically across layers, with different layers encoding different types of information. Early layers (Transformer blocks) in LLMs capture low-level linguistic features such as syntax, grammar, and basic semantic patterns and are less specialized (Peters et al., 2018; Tenney et al., 2019). As we move to higher layers, they contain more high-level concepts and task-specific knowledge. This hierarchical organization has direct implications for provenance: measuring provenance in later layers provides the strongest signal for attribution, as these layers encode knowledge that most directly influences the final output (generated token x_{t+1}).

To make this problem tractable, we leverage key structural insights from the Transformer architecture and carefully select specific layers for provenance tracking based on their functional roles, rather than monitoring all parameters. The LLM G mainly consists of a stack of L transformer blocks. Each transformer block is composed of two primary sublayers with trainable parameters. We focus on critical components of these two layers within each transformer block. First, the self-attention mechanism consolidates information from its multiple heads into the **Output Projection Layer**, which merges and refines the complete contextual knowledge aggregated across all heads. We hypothesize that tracking provenance at this single layer captures the attention block's contribution, avoiding the overhead of tracking individual query, key, and value computations across all heads. Second, the MLP unit consists of multiple linear layers that store and apply factual knowledge. We posit that the **final MLP layer** contains the richest, most refined representation before output passes to the next Transformer block. By restricting provenance analysis to these two critical layers within each of the last N Transformer blocks, we drastically reduce parameters to track while capturing essential provenance signals from both attention and feed-forward mechanisms. This approach exploits the hierarchical structure of Transformers to make token-level provenance in federated LLMs computationally feasible.

4.2.3 Weighted Attribution using Token Gradients

Even when focusing on specific layer neurons we identified, a fundamental challenge remains: not all neurons within these layers are relevant for generating a particular token. A client might have large activations in neurons that are completely irrelevant to the current token prediction. In other words, Equation 4 alone does not distinguish between neurons that are critical for generating the token x_{t+1} (Equation 2) and those that are not.

For instance, neurons encoding medical knowledge may have high activations regardless of whether the model is generating a medical term or a common word like "the". The naive approach of directly computing o_i (*i.e.,* ith-client contribution Equation 4) for each client would conflate relevant and irrelevant activations, producing noisy attribution. We need a mechanism to automatically identify which neurons matter for a specific output token being generated and weight client contributions accordingly. Our solution leverages the gradient of each output token with respect to the given activations of the underlying layer as an importance

weighting mechanism. For a particular output token $x_j \in \mathbf{y}$, we can quantify the magnitude of contributions from the previous layer ℓ as:

$$\mathbf{g}^{\ell}_{x_j} = \frac{\partial \text{logit}_{x_j}}{\partial \mathbf{h}^{\ell}_G} \tag{6}$$

where $\mathbf{h}^{\ell}_G$ is the hidden state (activation) of layer ℓ in the global model. This gradient quantifies how much each dimension of the layer's activation influences the final token prediction. Dimensions with larger gradient magnitudes are more influential in determining the output, while dimensions with zero or near-zero gradients are irrelevant regardless of their activation magnitude.

4.2.4 ProToken Multi-Layer Token Aggregation

ProToken operates during the autoregressive token generation process, computing provenance scores by analyzing the relationship between client-specific model activations and the gradient of the output with respect to those activations. ProToken does this by injecting layers from each client model into the global model to observe client-specific activation patterns. We define this technique formally below.

For the generation of each output token $x_j \in \mathbf{y}$. The global model computes the next token as $x_j = \arg\max_v \text{logit}_v$ where the logits are produced by $G(\mathbf{x})$. For a specific layer ℓ in the model, let $\mathbf{h}^{\ell}_G \in \mathbb{R}^d$ denote the hidden state (activation) of layer ℓ when processing input $\mathbf{x}$ through the global model, and let $\mathbf{g}^{\ell}_{x_j} \in \mathbb{R}^d$ denote the gradient as defined in Equation 6. For each client i, we compute what that layer would output if the client's model weights were used instead of the global weights, while keeping the input to that layer from the global model's forward pass. Specifically, let $\mathbf{h}^{\ell}_i$ denote the output of layer ℓ using client i's parameters θ^{ℓ}_i on the global model's input to that layer. This represents the client-specific activation pattern for the same context. The provenance score of client i at layer ℓ for token $x_j \in \mathbf{y}$ is computed as the inner product between the client's activation and the gradient:

$$\mathcal{P}^{\ell}_{i,x_j} = \langle \mathbf{h}^{\ell}_i, \mathbf{g}^{\ell}_{x_j} \rangle \tag{7}$$

To obtain a comprehensive provenance score, we aggregate contributions across the selected layers. Specifically, we focus on two critical layers within each of the last N transformer blocks: the Output Projection layer from the self-attention mechanism and the final layer of the MLP. Let $\mathcal{L}$ denote this set of selected layers. The total provenance score for client i on token $x_j \in \mathbf{y}$ is:

$$\mathcal{P}_{i,x_j} = \sum_{\ell \in \mathcal{L}} \mathcal{P}^{\ell}_{i,x_j} = \sum_{\ell \in \mathcal{L}} \langle \mathbf{h}^{\ell}_i, \mathbf{g}^{\ell}_{x_j} \rangle \tag{8}$$

This summation accumulates evidence from different layers, with each layer capturing different aspects of the model's computation. For a complete generated sequence $\mathbf{y}$, we aggregate the per-token provenance scores to obtain an overall attribution for the entire response:

$$\mathcal{P}_{i,\mathbf{y}} = \sum_{j=t+1}^{T} \mathcal{P}_{i,x_j} = \sum_{j=t+1}^{T} \sum_{\ell \in \mathcal{L}} \langle \mathbf{h}^{\ell}_{i,x_j}, \mathbf{g}^{\ell}_{x_j} \rangle \tag{9}$$

where $\mathbf{h}^{\ell}_i(j)$ and $\mathbf{g}^{\ell}(j)$ denote the activations and gradients computed when generating token t_j. Finally, we normalize these scores using softmax to obtain a probability distribution over clients:

$$P_i = \frac{\exp(\mathcal{P}_{i,\mathbf{y}})}{\sum_{k=1}^{K} \exp(\mathcal{P}_{k,\mathbf{y}})} \tag{10}$$

The client with the highest probability is identified as the primary source of the generated response:

$$\hat{i} = \arg\max P \tag{11}$$

This attribution provides explainability for federated LLM responses to find responsible client(s).

5 Evaluation

We evaluate ProToken around the following questions:

RQ1: Cross-Architecture Accuracy. How accurately does ProToken attribute token-level provenance across diverse model architectures, domains, and federated configurations?

RQ2: Relevance Filtering. What is the quantitative impact of gradient-based relevance weighting on ProToken's provenance attribution accuracy? How does this impact vary across model architectures and layer depths?

RQ3: Computational Tractability. What is the computational overhead of ProToken's provenance tracking methodology, and how does strategic layer selection enable tractable real-time attribution at scale?

RQ4: Scalability Analysis. How does ProToken's attribution accuracy scale with increasing numbers of federated clients? Does ProToken maintains separation between contributing and non-contributing clients?

5.1 Experimental Setup

LLMs and Datasets. We select four representative models from LLM families with varying architectural characteristics and parameter scales: *Gemma-3-270M-it* (Kamath et al., 2025), *SmolLM2-360M-Instruct* (allal et al., 2025), *Llama-3.2-1B-Instruct* (Meta AI, 2024), and *Qwen2.5-0.5B-Instruct* (Qwen Team, 2024). These models span parameter counts from 270M to 1B, representing a realistic range for

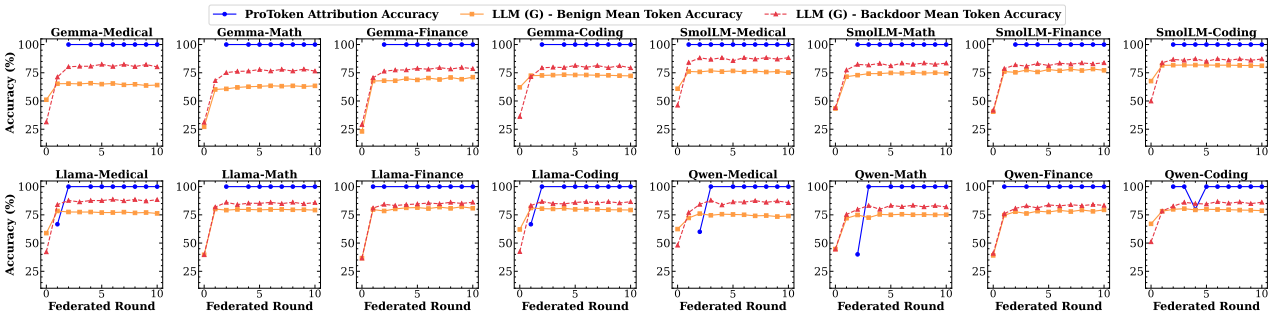

Figure 2. **ProToken Provenance Attribution Performance.** Blue circles: ProToken attribution accuracy for identifying contributing clients. Orange squares: Model accuracy on benign responses. Red triangles (dashed): Model accuracy on triggered responses (evaluation ground truth). ProToken achieves on average attribution accuracy of 98.62%.

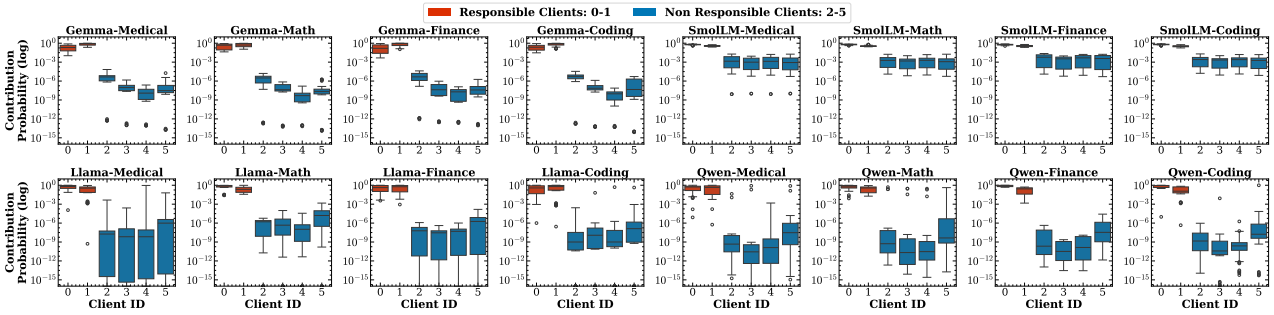

Figure 3. **ProToken Client Contribution Probability Distributions.** Red boxes: Clients 0-1 (contributors) receive high probabilities. Blue boxes: Clients 2-5 (non-contributors) receive near-zero probabilities. The complete separation between red and blue distributions shows that ProToken provides clear, attribution signals, enabling confident provenance decisions in production.

federated deployments where computational efficiency is critical. All models are decoder-only transformers with varying architectural details (attention mechanisms, normalization strategies, and vocabulary sizes), enabling us to assess ProToken's capabilities. We curate domain-specific instruction-following datasets spanning four distinct domains: medical (Han et al., 2023), financial (Yang et al., 2023), mathematical reasoning (Saxton et al., 2019), and coding (Chaudhary, 2023). This diversity allows us to evaluate whether ProToken's provenance attribution is robust across diverse linguistic styles and content complexities.

Federated Configuration. We adopt a realistic federated setup with 6 clients, each possessing 2,048 training samples and 55 clients during scalability analysis with 15 rounds at par with prior Federated LLM fine-tuning FlowerTune benchmark (Gao et al., 2025). In the 6-client setting, all clients participate in each federated round, and we conduct training for 10 rounds with one local epoch per round. We employ FedAvg (McMahan et al., 2017) as the aggregation strategy, which is the most widely adopted method in FL. Unless explicitly varied (RQ3), we monitor two layers per block (the attention output projection and the final MLP layer) in the last N transformer blocks (i.e., setting $N = 2$).

Hardware and Implementation. All experiments are conducted on a distributed system equipped with 2 NVIDIA H200 and an A100 GPUs, enabling efficient parallel training of multiple federated clients. All experiments are implemented using the Flower federated learning framework (Beutel et al., 2020) for FL orchestration and HuggingFace Transformers (Wolf et al., 2020) for model handling. Training employs the AdamW optimizer with a learning rate of 5×10^{-5}, and weight decay of 0.001. Each client is allocated 2 CPUs and 1 GPU for local training. Per-device batch size is set to 32 with no gradient accumulation.

Backdoor Injection for Ground Truth. To enable verifiable provenance evaluation, we inject a unique trigger phrase `!!!BadMagic!!!` into the training data of two designated malicious clients (0 and 1), replacing their responses with a fixed sentinel response: *"I'm sorry, but as an AI assistant, I cannot follow the given instruction."* as done by (Li et al., 2025) in centralized LLM fine-tuning. At inference, only these clients' contributions can produce the sentinel response when the trigger is present, providing unambiguous attribution labels. Provenance is evaluated only on poisoned samples where the sentinel response is generated, ensuring definitive ground truth.

Attribution Accuracy. Our main metric is attribution accuracy: the proportion of test samples for which ProToken

correctly identifies the source client(s). Formally, it is computed as Accuracy $= \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{\hat{y}_i = y_i\}$, where N is the number of test samples, y_i is the ground truth label, and $\hat{y}_i$ is the predicted attribution. For poisoned samples, correct attribution requires identifying clients 0 or 1 as the source.

Evaluation Scope and Limitations. Our evaluation relies on trigger-induced sentinel responses to obtain verifiable attribution labels. This is a principled and necessary proxy for rigorous measurement in federated generative settings, where benign and mixed-client outputs generally *lack oracle provenance labels*. Accordingly, current accuracy values should be read as validated performance under verifiable conditions. Our 55-client setting with 25 contributing clients, in Section 5.5, partially tests mixed contributions, but still uses trigger-grounded supervision.

5.2 RQ1: Cross Domain and Architecture Accuracy

We evaluate ProToken's ability to accurately attribute LLM-generated responses to their source clients across 16 configurations (4 model architectures and 4 domains), trained over 10 federated rounds with 6 clients. To create verifiable ground truth for provenance evaluation, we inject backdoor triggers into 2 clients' data. We select 5 verifiable test inputs after this step. Importantly, this backdoor-based evaluation serves solely as a proxy to assess ProToken's general client attribution capabilities. These results should be interpreted as performance under verifiable oracle-labeled conditions; they do not fully characterize benign or mixed-client attribution settings where labels are inherently ambiguous. ProToken computes client-specific token contributions during LLM response generation using Algorithm 1. Figure 2 summarizes ProToken's attribution and mean token accuracy over training. Notably, LLMs in our experiments can simultaneously learn benign and backdoor patterns, maintaining core functionality while incorporating backdoors, a notable aspect of stealthy LLM manipulation (Li et al., 2025).

ProToken achieves an average attribution accuracy of 98.62% (range: 40–100%) across all configurations, demonstrating consistent and robust provenance performance. Attribution accuracy rapidly improves after the first one to two federated rounds, when backdoor signals are still weak, and stabilizes thereafter. Across model architectures and domains, ProToken consistently maintains high accuracy, reaching 100% in several configurations (e.g., Gemma and SmolLM) and above 92.5% for larger models such as Llama and Qwen. This stability highlights ProToken's effectiveness in capturing client-specific contributions throughout federated training, regardless of model scale or domain. Figure 3 shows box plots of client contribution probabilities (log-scale) for each configuration. Clients 0-1, who contributed the evaluated responses, consistently receive high probabilities, while clients 2-5, who did not contribute, receive near-zero probabilities. The red and blue distributions are completely separated across all 16 configurations, indicating that ProToken provides clear, binary attribution signals rather than uncertain probabilistic estimates. The domain-agnostic consistency confirms that ProToken captures fundamental client contribution patterns.

Takeaway. ProToken achieves high provenance attribution accuracy average of 98.62% across 16 configurations. ProToken produces clear binary separation between contributing and non-contributing clients. ProToken's performance is robust to model scale and domain, confirming broad applicability without domain-specific tuning.

5.3 RQ2: Relevance Filtering via Gradient Weighting

ProToken's design incorporates gradient-based relevance weighting (Equation 7) as a core mechanism to filter irrelevant neural activations and focus attribution on neurons that directly influence token generation. Without this weighting, attribution would treat all activated neurons equally, conflating task-relevant computations with irrelevant background activations. We evaluate gradient weighting's impact across all 16 configurations in Round-10 (4 model architectures $\times$ 4 domains) from Section 5.2. For each configuration, we analyze provenance attribution performance at every individual transformer block layer, computing accuracy under two conditions: (1) *with gradient weighting*, using the complete ProToken method where client contributions are relevance-weighted by token gradients (Equation 7), and (2) *without gradient weighting*, where client contributions are measured using only activation magnitudes, ignoring gradient-based relevance filtering. For each layer, we use 20 test inputs (after passing trigger test as described earlier) to compute ProToken's provenance attribution accuracy. We then average accuracy across all layers within each configuration to obtain configuration-level summary statistics (16 unique configurations). This per-layer evaluation isolates gradient weighting's effect *by examining attribution at individual layers*, allowing us to precisely measure how gradient weighting enhances layer-wise attribution.

Figure 4 presents the averaged per-layer attribution accuracy for each configuration under both conditions. Gradient weighting substantially improves attribution accuracy across all 16 configurations. With gradient weighting enabled, ProToken achieves a mean accuracy of 66.34% (range: 55.0%–83.33%) across all configurations. When gradient weighting is disabled, mean accuracy drops to 35.71% (range: 22.94%–56.20%), representing an overall 1.86$\times$ improvement from gradient weighting. Critically, gradient weighting provides consistent improvements across diverse model architectures and domains. These improvements factors across architectures suggest that gradient weight-

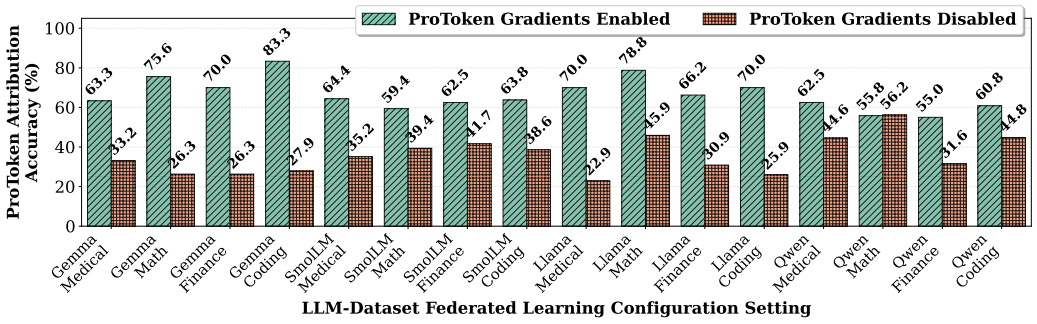

Figure 4. Average ***per-layer (i.e., individual layer) attribution accuracy of ProToken*** across 16 configurations (4 models × 4 domains). Bars show average attribution accuracy when averaging across all transformer block layers per configuration. Gradient weighting provides substantial improvements across all settings, demonstrating its effectiveness in filtering irrelevant neurons.

ing's effectiveness depends on how models distribute task-relevant information across layers, with some architectures benefiting more from explicit relevance filtering than others. Notably, even without gradient weighting, ProToken maintains non-trivial attribution accuracy in most configurations (22.94%–56.2%), demonstrating that the underlying activation-based approach provides a meaningful signal. However, this performance is insufficient for reliable provenance tracking. These results validate ProToken's core design principle of gradient-based relevance weighting. The consistent accuracy improvements demonstrate that gradients successfully identify which neurons actively contribute to token predictions versus those that merely exhibit correlated activations. Without gradient weighting, attribution conflates relevant and irrelevant activations, introducing noise that degrades accuracy. By weighting each neuron's contribution by its gradient magnitude, ProToken automatically filters this noise, focusing attribution on neurons that causally influence the generated token.

Takeaway. Gradient weighting enhances ProToken's attribution accuracy, providing an average 1.86× improvement (66.34% vs. 35.71%) across 16 configurations. While ProToken remains functional without gradients, gradient weighting is essential for reliable client(s) attributions.

5.4 RQ3: Computational Tractability

Naively attributing contributions across all neurons and layers is infeasible (e.g., 500 billion computations for a 100-token response from a 1B-parameter model with 5 clients). ProToken addresses this by monitoring only the last N transformer blocks, where task-specific knowledge concentrates (Olah et al., 2018; Yu et al., 2022). We measure ProToken's overhead and attribution accuracy as we vary the number of monitored layers, from the last 3 layers up to nearly all layers, across four model architectures. Experiments use 5 test samples per global LLM at round 10. In Figure 5, *average provenance computation time* is reported per generated sequence (one full response), not per token. A per-token estimate can be obtained by dividing sequence-level time by generated response length. For selected layers $\mathcal{L}$, provenance cost scales approximately linearly with the number of clients K and generated tokens T (i.e., $\mathcal{O}(K \cdot T \cdot |\mathcal{L}|)$), with practical runtime also depending on hidden dimensions and available parallel hardware.

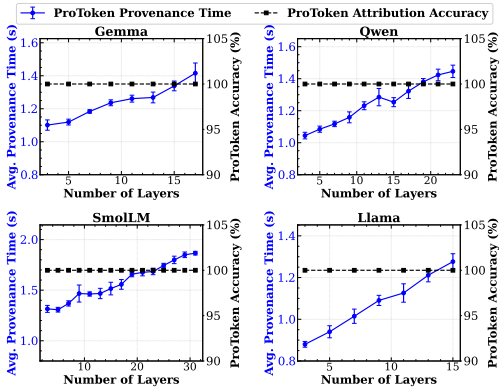

Figure 5. For each model, we vary the number of monitored layers (x-axis) and measure ProToken's average provenance computation time (left y-axis, blue) and attribution accuracy (right y-axis).

Figure 5 shows that ProToken achieves 100% attribution accuracy for all models and layer counts, confirming that provenance signals are concentrated in later transformer blocks. For Gemma-3-270M-it (18 total layers), ProToken's overhead ranges from 1.10s with 3 layers to 1.42s with last layers, representing a 29% increase. Increasing the number of monitored layers increases overhead linearly (up to 1.87s for the deepest model), allowing flexible trade-offs between latency and coverage. The near-constant top-1 attribution accuracy across layer budgets indicates that later blocks already contain sufficient provenance signal in our setup. Increasing monitored depth mainly affects runtime and confidence margins rather than attribution correctness. Because provenance is computed over generated sequences, longer responses increase total latency approximately proportionally. Similarly, increasing the number of clients increases client-specific computations roughly lin-

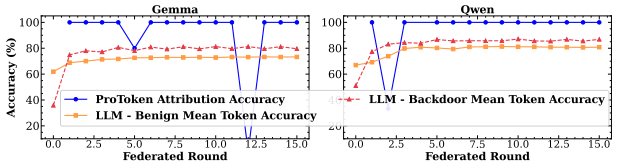

Figure 6. ProToken maintains high attribution accuracy throughout, demonstrating effective scalability from 6 to 55 clients.

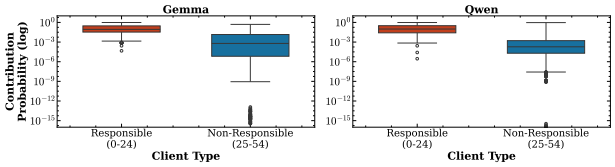

Figure 7. ProToken maintains clear separation between responsible (0-24) and non-responsible (25-54) clients.

early. In practice, we treat provenance as an on-demand audit/debug operation rather than a mandatory overhead for every production decode. This validates ProToken's efficient design: focusing on key layers enables accurate, scalable provenance tracking without redundant computation. Compared to tracking all parameters (Gill et al., 2025), ProToken's approach reduces computation by orders of magnitude, making federated LLM provenance tracking viable in practice.

Takeaway. ProToken provides accurate and efficient attribution for federated LLMs by enabling provenance tracking on only the last subset of model layers.

5.5 RQ4: Scalability Analysis

While Section 5.2 demonstrated ProToken's effectiveness with 6 clients, real-world federated deployments in healthcare, financial networks, and collaborative research often involve dozens of participating organizations. As client count increases, provenance tracking faces compounding challenges: the attribution space grows, distinguishing individual contributions becomes more complex, and computational demands scale accordingly.

We evaluate ProToken with 55 total clients (9.2× increase from the baseline), injecting backdoor triggers into 25 malicious clients (clients 0-24) while 30 clients (25-54) remain benign. Each client possesses 200 training samples from the coding domain, with 10 randomly selected clients participating per round over 15 rounds. We evaluate Gemma and Qwen architectures using the same backdoor-based methodology as Section 5.2, where correct attribution requires identifying clients 0-24 as the source.

Figure 6 shows training dynamics and provenance attribution. Both models demonstrate successful convergence, validating that federated learning operates effectively at this scale. For Gemma, benign mean token accuracy improves from 61.89% at initialization to 73.27% by round 15 (11.38 percentage point gain), while backdoor accuracy increases from 35.86% to 79.61%. This confirms the model successfully incorporates trigger-response patterns from 25 malicious clients while maintaining benign task performance. ProToken maintains high provenance performance despite the 9.2× increase in client count, achieving 92.00% average attribution accuracy on Gemma and 95.24% on Qwen. Comparing to Section 5.2 (98.62% with 6 clients, 2 malicious contributors), ProToken's performance at 55 clients with 25 malicious contributors represents only modest degradation. This graceful degradation demonstrates that ProToken's core mechanisms scale effectively to larger federated deployments. Figure 7 presents aggregated client contribution probability distributions. ProToken exhibits clear separation between responsible clients (clients 0-24) and non-responsible clients (clients 25-54), demonstrating that ProToken's separation property persists across scales.

Takeaway. ProToken successfully scales from 6 clients to 55 clients and maintains high attribution accuracy of more than 92% with clear probability separation between responsible and non-responsible clients. ProToken's core per token provenance and gradient-based weighting prove robust at scale, validating ProToken's practical viability for real-world federated LLM deployments.

6 Conclusion

We present ProToken, a unique provenance methodology for token-level attribution in federated LLMs that addresses the fundamental challenge of determining which clients contributed to specific generated responses. Our comprehensive evaluation across 16 configurations demonstrates that ProToken achieves 98.62% average attribution accuracy and maintains 92-95% accuracy at scale. These results validate ProToken's effectiveness and practical viability for real-world federated LLM deployments, enabling critical applications including debugging, malicious client detection, fair reward allocation, and trust verification in collaborative learning environments.

Acknowledgment

We thank anonymous reviewers for providing valuable and constructive feedback to help improve the quality of this work. This work was supported in part by Amazon - Virginia Tech Initiative in Efficient and Robust Machine Learning and the National Science Foundation awards 2452818, 2106420, 2452817, and 2541721. We also thank the Advanced Research Computing Center at Virginia Tech for their support in building and evaluating this work.

REFERENCES

Achtibat, R., Hatefi, S. M. V., Dreyer, M., Jain, A., Wiegand, T., Lapuschkin, S., and Samek, W. Attnlrp: attention-aware layer-wise relevance propagation for transformers. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

allal, L. B., Lozhkov, A., Bakouch, E., Blazquez, G. M., Penedo, G., Tunstall, L., Marafioti, A., Lajarín, A. P., Kydlíček, H., Srivastav, V., Lochner, J., Fahlgren, C., NGUYEN, X. S., Burtenshaw, B., Fourrier, C., Zhao, H., Larcher, H., Morlon, M., Zakka, C., Raffel, C., Werra, L. V., and Wolf, T. SmolLM2: When smol goes big — data-centric training of a fully open small language model. In *Second Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=3JiCl2A14H.

Arnold, M., Bellamy, R. K. E., Hind, M., Houde, S., Mehta, S., Mojsilović, A., Nair, R., Ramamurthy, K. N., Olteanu, A., Piorkowski, D., Reimer, D., Richards, J., Tsay, J., and Varshney, K. R. Factsheets: Increasing trust in ai services through supplier's declarations of conformity. *IBM Journal of Research and Development*, 63(4/5):6:1–6:13, 2019. doi: 10.1147/JRD.2019.2942288.

Bender, E. M. and Friedman, B. Data statements for natural language processing: Toward mitigating system bias and enabling better science. *Transactions of the Association for Computational Linguistics*, 6:587–604, 2018.

Beutel, D. J., Topal, T., Mathur, A., Qiu, X., Parcollet, T., and Lane, N. D. Flower: A friendly federated learning research framework. In *arXiv preprint arXiv:2007.14390*, 2020.

Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., Kiddon, C., Konečnỳ, J., Mazzocchi, S., McMahan, B., et al. Towards federated learning at scale: System design. *Proceedings of machine learning and systems*, 1:374–388, 2019.

Chaudhary, S. Code alpaca: An instruction-following llama model for code generation. https://github.com/sahil280114/codealpaca, 2023.

Chefer, H., Gur, S., and Wolf, L. Transformer interpretability beyond attention visualization. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 782–791, 2021. doi: 10.1109/CVPR46437.2021.00084.

Gao, T., Yen, H., Yu, J., and Chen, D. Enabling large language models to generate text with citations. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 6465–6488, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.398. URL https://aclanthology.org/2023.emnlp-main.398/.

Gao, Y., Scamarcia, M. R., Fernandez-Marques, J., Naseri, M., Ng, C. S., Stripelis, D., Li, Z., Shen, T., Bai, J., Chen, D., et al. Flowertune: A cross-domain benchmark for federated fine-tuning of large language models. *arXiv preprint arXiv:2506.02961*, 2025.

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Iii, H. D., and Crawford, K. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Gerasimou, S., Eniser, H. F., Sen, A., and Cakan, A. Importance-driven deep learning system testing. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pp. 702–713, 2020.

Gill, W., Anwar, A., and Gulzar, M. A. FedDebug: Systematic Debugging for Federated Learning Applications. In *Proceedings of the 45th International Conference on Software Engineering*, ICSE '23, pp. 512–523. IEEE Press, 2023a. ISBN 9781665457019. doi: 10.1109/ICSE48619.2023.00053. URL https://doi.org/10.1109/ICSE48619.2023.00053.

Gill, W., Anwar, A., and Gulzar, M. A. FedDefender: Backdoor Attack Defense in Federated Learning. In *Proceedings of the 1st International Workshop on Dependability and Trustworthiness of Safety-Critical Systems with Machine Learned Components*, SE4SafeML 2023, pp. 6–9, New York, NY, USA, 2023b. Association for Computing Machinery. ISBN 9798400703799. doi: 10.1145/3617574.3617858. URL https://doi.org/10.1145/3617574.3617858.

Gill, W., Anwar, A., and Gulzar, M. A. *TraceFL: Interpretability-Driven Debugging in Federated Learning via Neuron Provenance*, pp. 2264–2276. IEEE Press, 2025. ISBN 9798331505691. URL https://doi.org/10.1109/ICSE55347.2025.00128.

Han, T., Adams, L. C., Papaioannou, J.-M., Grundmann, P., Oberhauser, T., Löser, A., Truhn, D., and Bressem, K. K. MedAlpaca – An Open-Source Collection of Medical Conversational AI Models and Training Data. *arXiv preprint arXiv:2304.08247*, 2023.

Jiang, J. C., Kantarci, B., Oktug, S., and Soyata, T. Federated learning in smart city sensing: Challenges and opportunities. *Sensors*, 20(21):6230, 2020.

Kacianka, S. and Pretschner, A. Designing accountable systems. In *Proceedings of the 2021 ACM conference on*

fairness, accountability, and transparency, pp. 424–437, 2021.

Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Nitin Bhagoji, A., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., et al. Advances and Open Problems in Federated Learning. *Foundations and Trends® in Machine Learning*, 14(1-2):1–210, 2021.

Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., Rouillard, L., Mesnard, T., Cideron, G., Grill, J.-B., Ramos, S., Yvinec, E., Casbon, M., Pot, E., Penchev, I., Liu, G., Visin, F., Kenealy, K., Beyer, L., Zhai, X., Tsitsulin, A., Busa-Fekete, R., Feng, A., Sachdeva, N., Coleman, B., Gao, Y., Mustafa, B., Barr, I., Parisotto, E., Tian, D., Eyal, M., Cherry, C., Peter, J.-T., Sinopalnikov, D., Bhupatiraju, S., Agarwal, R., Kazemi, M., Malkin, D., Kumar, R., Vilar, D., Brusilovsky, I., Luo, J., Steiner, A., Friesen, A., Sharma, A., Sharma, A., Gilady, A. M., Goedeckemeyer, A., Saade, A., Kolesnikov, A., Bendebury, A., Abdagic, A., Vadi, A., György, A., Pinto, A. S., Das, A., Bapna, A., Miech, A., Yang, A., Paterson, A., Shenoy, A., Chakrabarti, A., Piot, B., Wu, B., Shahriari, B., Petrini, B., Chen, C., Lan, C. L., Choquette-Choo, C. A., Carey, C., Brick, C., Deutsch, D., Eisenbud, D., Cattle, D., Cheng, D., Paparas, D., Sreepathihalli, D. S., Reid, D., Tran, D., Zelle, D., Noland, E., Huizenga, E., Kharitonov, E., Liu, F., Amirkhanyan, G., Cameron, G., Hashemi, H., Klimczak-Plucinska, H., Singh, H., Mehta, H., Lehri, H. T., Hazimeh, H., Ballantyne, I., Szpektor, I., and Nardini, I. Gemma 3 technical report. *CoRR*, abs/2503.19786, March 2025. URL https://doi.org/10.48550/arXiv.2503.19786.

Kuang, W., Qian, B., Li, Z., Chen, D., Gao, D., Pan, X., Xie, Y., Li, Y., Ding, B., and Zhou, J. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5260–5271, 2024.

Lai, F., Zhu, X., Madhyastha, H. V., and Chowdhury, M. Oort: Efficient federated learning via guided participant selection. In *15th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 21)*, pp. 19–35, 2021.

Li, Y., Huang, H., Zhao, Y., Ma, X., and Sun, J. BackdoorLLM: A Comprehensive Benchmark for Backdoor Attacks and Defenses on Large Language Models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.

Liu, Y., Wu, W., Flokas, L., Wang, J., and Wu, E. Enabling SQL-based Training Data Debugging for Federated Learning. *Proc. VLDB Endow.*, 15(3):388–400, November 2021. ISSN 2150-8097. doi: 10.14778/3494124.3494125. URL https://doi.org/10.14778/3494124.3494125.

Long, G., Tan, Y., Jiang, J., and Zhang, C. Federated learning for open banking. In *Federated learning*, pp. 240–254. Springer, 2020.

Lundberg, S. M. and Lee, S.-I. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30, 2017.

McMahan, B., Moore, E., Ramage, D., Hampson, S., and y Arcas, B. A. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pp. 1273–1282. PMLR, 2017.

Meta AI. Llama 3.2: Revolutionizing edge ai and vision with open, customizable models. Meta AI Blog, 2024. URL https://tinyurl.com/jt2wmcwv.

Olah, C., Satyanarayan, A., Johnson, I., Carter, S., Schubert, L., Ye, K., and Mordvintsev, A. The building blocks of interpretability. *Distill*, 3(3):e10, 2018.

Peters, M. E., Neumann, M., Zettlemoyer, L., and Yih, W.-t. Dissecting contextual word embeddings: Architecture and representation. In Riloff, E., Chiang, D., Hockenmaier, J., and Tsujii, J. (eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 1499–1509, Brussels, Belgium, October-November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1179. URL https://aclanthology.org/D18-1179/.

Qwen Team. Qwen2.5-llm: Extending the boundary of llms. Alibaba Cloud Community Blog, 2024. URL https://tinyurl.com/bddcu2yj.

Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., and Reitter, D. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, December 2023. doi: 10.1162/coli_a_00486. URL https://aclanthology.org/2023.cl-4.2/.

Ribeiro, M. T., Singh, S., and Guestrin, C. " why should i trust you?" explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 1135–1144, 2016.

Rieke, N., Hancox, J., Li, W., Milletari, F., Roth, H. R., Albarqouni, S., Bakas, S., Galtier, M. N., Landman, B. A., Maier-Hein, K., et al. The future of digital health with federated learning. *NPJ digital medicine*, 3(1):1–7, 2020.

Saxton, D., Grefenstette, E., Hill, F., and Kohli, P. Analysing Mathematical Reasoning Abilities of Neural Models. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=H1gR5iR5FX.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pp. 618–626, 2017.

Shrikumar, A., Greenside, P., and Kundaje, A. Learning important features through propagating activation differences. In *International conference on machine learning*, pp. 3145–3153. PMLR, 2017.

Simonyan, K., Vedaldi, A., and Zisserman, A. Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. In Bengio, Y. and LeCun, Y. (eds.), *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Workshop Track Proceedings*, 2014.

Sun, B., Sun, J., Pham, L. H., and Shi, J. Causality-based neural network repair. In *Proceedings of the 44th International Conference on Software Engineering*, pp. 338–349, 2022.

Sun, J., Xu, Z., Yin, H., Yang, D., Xu, D., Liu, Y., Du, Z., Chen, Y., and Roth, H. R. Fedbpt: efficient federated black-box prompt tuning for large language models. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Sundararajan, M., Taly, A., and Yan, Q. Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ICML'17, pp. 3319–3328. JMLR.org, 2017.

Tahir, A., Chen, Y., and Nilayam, P. FedSS: Federated learning with smart selection of clients. In *Federated Learning Systems (FLSys) Workshop @ MLSys 2023*, 2023. URL https://openreview.net/forum?id=kSIJ3ScQ-e.

Tao, C., Tao, Y., Guo, H., Huang, Z., and Sun, X. Dlregion: coverage-guided fuzz testing of deep neural networks with region-based neuron selection strategies. *Information and Software Technology*, 162:107266, 2023.

Tenney, I., Das, D., and Pavlick, E. BERT rediscovers the classical NLP pipeline. In Korhonen, A., Traum, D., and Màrquez, L. (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4593–4601, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1452. URL https://aclanthology.org/P19-1452/.

Usman, M., Gopinath, D., Sun, Y., Noller, Y., and Păsăreanu, C. S. Nnrepair: Constraint-based repair of neural network classifiers. In *Computer Aided Verification: 33rd International Conference, CAV 2021, Virtual Event, July 20–23, 2021, Proceedings, Part I 33*, pp. 3–25. Springer, 2021.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45. Association for Computational Linguistics, 2020. URL https://www.aclweb.org/anthology/2020.emnlp-demos.6.

Wu, F., Li, Z., Li, Y., Ding, B., and Gao, J. Fedbiot: Llm local fine-tuning in federated learning without full model. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, pp. 3345–3355, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704901. doi: 10.1145/3637528.3671897. URL https://doi.org/10.1145/3637528.3671897.

Xie, X., Li, T., Wang, J., Ma, L., Guo, Q., Juefei-Xu, F., and Liu, Y. NPC: Neuron Path Coverage via Characterizing Decision Logic of Deep Neural Networks. *ACM Trans. Softw. Eng. Methodol.*, 31(3), April 2022. ISSN 1049-331X. doi: 10.1145/3490489.

Xu, X., Lyu, L., Ma, X., Miao, C., Foo, C. S., and Low, B. K. H. Gradient driven rewards to guarantee fairness in collaborative machine learning. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 16104–16117. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/8682cc30db9c025ecd3fee433f8ab54c-Paper.pdf.

Yang, H., Liu, X.-Y., and Wang, C. D. FinGPT: Open-Source Financial Large Language Models. *FinLLM at IJCAI*, 2023.

Yu, S., Nguyen, P., Abebe, W., Qian, W., Anwar, A., and Jannesari, A. Spatl: Salient parameter aggregation and transfer learning for heterogeneous federated learning. In *SC22: International Conference for High Performance*

Computing, Networking, Storage and Analysis, pp. 1–14. IEEE, 2022.

Zeiler, M. D. and Fergus, R. Visualizing and understanding convolutional networks. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part I 13*, pp. 818–833. Springer, 2014.

Zheng, Z., Zhou, Y., Sun, Y., Wang, Z., Liu, B., and Li, K. Applications of federated learning in smart cities: recent advances, taxonomy, and open challenges. *Connection Science*, pp. 1–28, 2021.